Lab3 – Fixed-Outline Floorplanning with Soft Blocks and Placement Constraints

Physical Design Automation, Fall 2026

Release: 2026/11/10 Deadline: 2026/11/23 23:59

1 Introduction

Floorplanning decides the shape and the location of every block of a chip inside a given outline. In a modern SoC many blocks have a flexible shape with only an area budget, some have a fixed shape, some are already placed, and designers attach placement constraints such as "these blocks must have the same shape" or "this block must touch the left edge". This lab is a simplified variant of Problem C of the ICCAD 2026 CAD Contest (*The FloorSet Challenge: Data-Driven SoC Floorplanning*, Intel):

- the outline is fixed and given in the input;
- blocks are soft, fixed-shape, or preplaced rectangles, and no block may be rotated;
- the placement constraints are MIB groups and boundary constraints;
- every coordinate and dimension is an integer.

Implement a C++ floorplanner that places every block inside the outline without overlap, minimizes the wirelength, and satisfies as many MIB groups and boundary constraints as possible. Any method is allowed. There is no ranking between students: each test case is scored by the verifier against fixed thresholds (Section 6).

2 Problem Definition

Outline and blocks. The outline is a $W \times H$ rectangle with its lower-left corner at $(0, 0)$. A block is a rectangle with lower-left corner (x, y), width w, and height h; it occupies $[x, x + w] \times [y, y + h]$, and x, y, w, h are integers. Each block is one of:

- **Soft block:** only an area target a is given. You choose integer w, h subject to the area tolerance $100 \cdot |wh - a| \leq a$ and the aspect ratio bound $Rh \geq w$ and $Rw \geq h$ (that is, $1/R \leq h/w \leq R$; $R = 3$ in all released cases).
- **Fixed-shape block:** w, h are given; you choose the position.
- **Preplaced block:** w, h, x, y are all given.

A placement is *feasible* when every block lies inside the outline ($x \geq 0$, $y \geq 0$, $x + w \leq W$, $y + h \leq H$) and no two blocks overlap; blocks i, j overlap when $\max(x_i, x_j) < \min(x_i + w_i, x_j + w_j)$ and $\max(y_i, y_j) < \min(y_i + h_i, y_j + h_j)$. Touching is allowed. The *whitespace ratio* of a case is $1 - \big(\sum_{\text{soft}} a_b + \sum_{\text{fixed, preplaced}} w_b h_b\big)/(WH)$.

Wirelength. A terminal is a fixed point on the outline border. A connection joins two endpoints (block–block or block–terminal) with an integer weight c. The position of a block is its centroid

$(x + w/2,\ y + h/2)$; the position of a terminal is its own point. The wirelength is the weighted sum of Manhattan distances over all connections,

$$\mathrm{WL} = \sum_{\text{connections } (a,b,c)} c \cdot \big(|c_x(a) - c_x(b)| + |c_y(a) - c_y(b)|\big),$$

which is always a multiple of 0.5. The verifier computes it exactly.

Placement constraints. An *MIB group* is a set of soft blocks with identical area targets; it is satisfied when all members have identical (w, h) (their positions are independent). A *boundary constraint* requires a block to touch an outline edge:

`LEFT`: $x = 0$	`RIGHT`: $x + w = W$	`BOTTOM`: $y = 0$	`TOP`: $y + h = H$

A *corner constraint* lists two adjacent edges and requires both, that is, the block contains that corner of the outline. These are *scored constraints*: violating them keeps the output feasible but loses points.

3 Input Format

The input is read as whitespace-separated tokens, so line breaks are not significant. Sections appear in this order, and each count line is followed by exactly that many records. Text after `//` below is explanation only.

```
OUTLINE <W> <H>
WLTHRESHOLD <T>                          // wirelength threshold used for scoring
ASPECTRATIO <R>                          // aspect ratio bound of soft blocks
BLOCKS <n>
SOFT <name> <area>                       // one of these three record types
FIXED <name> <w> <h>                     //   per block, kinds in any order;
PREPLACED <name> <w> <h> <x> <y>         //   n records in total
TERMINALS <r>
TERMINAL <name> <x> <y>                  // r records
CONNECTIONS <m>
CONN <name1> <name2> <weight>            // m records
MIBS <q>
MIB <k> <name1> ... <namek>              // q records, k members each
BOUNDARIES <p>
BOUNDARY <name> <EDGE> [<EDGE>]          // p records; EDGE: LEFT RIGHT TOP BOTTOM
```

Every input file satisfies the following guarantees.

- All numbers are integers except T, a positive multiple of 0.5 printed with one decimal (for example `123456.5`). T is a reference wirelength obtained by the TA's floorplanner on that case. $R = 3$. Names contain no whitespace, are unique across blocks and terminals, and are never an edge keyword.
- Bounds: every coordinate and dimension is in $[0, 100000]$; $n \le 200$, $r \le 100$, $m \le 1000$, $q \le 20$, $p \le 100$, $1 \le \mathit{weight} \le 10$; $10000 \le \mathit{area} \le WH \le 10^{10}$. Coordinates fit in 32-bit integers, but areas, their products, and the wirelength can exceed 2^{31}: use 64-bit integers.
- Preplaced blocks lie strictly inside the outline and do not overlap each other. Terminals lie on the outline border.
- `CONN`: at least one endpoint is a block; never a block to itself; each unordered pair appears at most once.

- `MIB`: $k \geq 2$; members are soft blocks with identical area targets; a block is in at most one group. `BOUNDARY`: two edges are always adjacent; only soft and fixed-shape blocks; at most one record per block; at most 4 corner constraints per case.
- Every case has a known feasible solution that satisfies every MIB group and every boundary constraint.

4 Output Format

The output is read line by line: the first non-blank line is `WIRELENGTH <wl>`, the second is `BLOCKS <n>`, and each of the next n non-blank lines holds one block with exactly five tokens. Blank lines are ignored; a record split over several lines, or several records on one line, is rejected.

```
WIRELENGTH <wl>
BLOCKS <n>
<name> <x> <y> <w> <h>                  // n lines, in any order
```

`<wl>` is the wirelength of your placement as a plain decimal number (for example `1625.0`); it must match the verifier's value within 0.01, otherwise the WL score of the case is 0. `<n>` is the number of blocks in the input. `<x> <y>` is the lower-left corner and `<w> <h>` the size of the block; all four are integers. Every input block appears exactly once, and there are no unknown names.

5 Example

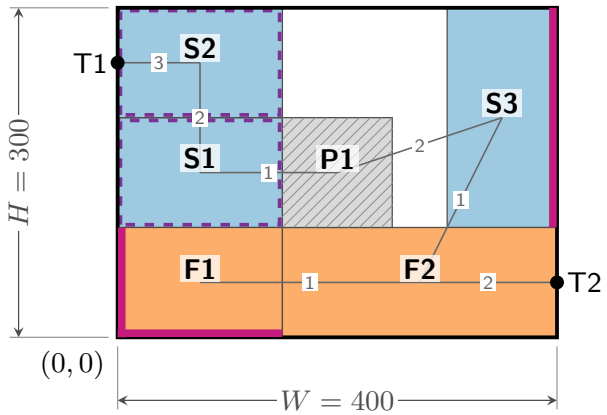
A toy case with six blocks (three soft, two fixed-shape, one preplaced), two terminals, seven connections, one MIB group $\{S1, S2\}$, and two boundary constraints: `F1 LEFT BOTTOM` (a corner) and `S3 RIGHT`.

Input

```
OUTLINE 400 300
WLTHRESHOLD 1700.0
ASPECTRATIO 3
BLOCKS 6
SOFT S1 15000
SOFT S2 15000
SOFT S3 20000
FIXED F1 150 100
FIXED F2 250 100
PREPLACED P1 100 100 150 100
TERMINALS 2
TERMINAL T1 0 250
TERMINAL T2 400 50
CONNECTIONS 7
CONN S1 S2 2
CONN S1 P1 1
CONN S2 T1 3
CONN P1 S3 2
CONN F1 F2 1
CONN F2 T2 2
CONN S3 F2 1
MIBS 1
MIB 2 S1 S2
BOUNDARIES 2
BOUNDARY F1 LEFT BOTTOM
BOUNDARY S3 RIGHT
```

Output

```
WIRELENGTH 1625.0
BLOCKS 6
S1 0 100 150 100
S2 0 200 150 100
S3 300 100 100 200
F1 0 0 150 100
F2 150 0 250 100
P1 150 100 100 100
```



Blue: soft blocks; orange: fixed-shape blocks; grey hatched: preplaced block. Dashed purple: the MIB group $\{S1, S2\}$; thick magenta bars: required edges of the boundary constraints. Dots: terminals; lines: connections between centroids, labeled with their weights.

S1 and S2 are both 150×100 (area 15000, ratio 2/3), S3 is 100×200; F1 touches $x = 0$ and $y = 0$, S3 touches $x + w = 400 = W$. The centroids are S1 $(75, 150)$, S2 $(75, 250)$, S3 $(350, 200)$, F1 $(75, 50)$, F2 $(275, 50)$, P1 $(200, 150)$, and the terminals are T1 $(0, 250)$, T2 $(400, 50)$. For example, `CONN P1 S3 2` contributes $2 \cdot (|200 - 350| + |150 - 200|) = 400$ and the block–terminal connection `CONN S2 T1 3` contributes $3 \cdot (|75 - 0| + |250 - 250|) = 225$; the seven connections sum to 1625. (Had S2 been 151×99 at the same corner, its centroid would be $(75.5, 249.5)$: this is how half-integer wirelengths arise.) The verifier report for this output is:

```
$ ./verifier toy.in toy.out
Gate checks:
  1. Output format          PASS
  2. Block names            PASS
  3. Positive size          PASS
  4. Fixed/preplaced shape  PASS
  5. Soft area tolerance    PASS
  6. Soft aspect ratio      PASS
  7. Inside outline         PASS
  8. Overlap-free           PASS
Result: FEASIBLE
Reported WL   1625.0
Computed WL   1625.0
Threshold T   1700.0
Gate          10.00 / 10
WL score      5.00 / 5
Boundary      3.00 / 3  (2/2 satisfied)
MIB           2.00 / 2  (1/1 satisfied)
Total 20.00 / 20
```

A feasible output that violates a scored constraint keeps the gate points and loses only that item: for instance, with S2 reshaped to 151×99 (and `WIRELENGTH 1628.0`) the MIB group is no longer satisfied and the case scores 18.00/20.

6 Scoring and Test Cases

There are 5 cases, 1 public (released with the assignment) and 4 hidden, each worth 20 points as in Table 1, for a total of 100 points. WL, boundary, and MIB are scored only on a feasible output. If $p = 0$ or $q = 0$, that item gives full points (released cases have $p, q > 0$).

The verifier prints each gate check as PASS or FAIL with the first violation (for example `Overlap: S3 P1`), then the reported WL, the computed WL, T, and the item scores. It exits with status 0 on a feasible output and 1 otherwise. The grading verifier is the same binary as the released one.

Hidden cases. The hidden cases follow the same format and guarantees as the public case: at most 200 blocks; every constraint type (soft, fixed-shape, and preplaced blocks, terminals, MIB groups, and boundary constraints including corners) appears in every case; whitespace ratio approximately 5 % or more; difficulty increases with the number of blocks, tighter whitespace, and higher constraint density. Do not tune your program to the public case only.

7 Execution

Provide a `Makefile` that builds an executable named `Lab3`. The TA compiles and runs your program, then checks the output with the released verifier (use `chmod +x verifier` the first time if needed):

```
make
./Lab3 <input_file> <output_file>
```

Table 1: Scoring of one case (20 points).

Item	Points	Rule
Gate	10	All of the following must hold; **any failure scores 0 for the whole case**. 1. The output parses (header lines, n block lines of five tokens, integers where required). 2. Every input block appears exactly once; no unknown names. 3. $w > 0$ and $h > 0$ for every block. 4. Fixed-shape blocks have their given (w, h); preplaced blocks their given (x, y, w, h). 5. Soft area tolerance $100 \cdot \lvert wh - a \rvert \le a$. 6. Soft aspect ratio $Rh \ge w$ and $Rw \ge h$. 7. Inside the outline. 8. Overlap-free (touching allowed). 9. Runtime at most 120 s and single-threaded execution. The verifier checks 1–8 and prints `FEASIBLE` exactly when they hold; check 9 is applied by the TA's grading script.
WL	5	0 if the reported `<wl>` differs from the computed WL by more than 0.01; otherwise, with T the `WLTHRESHOLD` of the case, $$S_{\mathrm{WL}} = \begin{cases} 5 & \mathrm{WL} \le T, \\ 5 \cdot \dfrac{1.05\,T - \mathrm{WL}}{0.05\,T} & T < \mathrm{WL} \le 1.05\,T, \\ 0 & \mathrm{WL} > 1.05\,T. \end{cases}$$
Boundary	3	$3 \cdot \mathit{satisfied}/p$; a block is satisfied only when it touches all of its listed edges.
MIB	2	$2 \cdot \mathit{satisfied}/q$; a group is satisfied when all members have identical (w, h).

```
./verifier <input_file> <output_file>
```

- Runtime limit: **120 seconds wall-clock per case**, measured on the course workstation (CPU: Intel Xeon E5-2630 v4 @ 2.20 GHz, OS: Rocky Linux 8.10, g++: 11.2.0). Exceeding it fails the gate.
- The program must be **single-threaded**: no `std::thread`, OpenMP, `fork`, or any other parallel work. A violation fails the gate.
- Compile and test your final version on the course workstation. A submission that fails to compile or run there receives 0 points.

8 Submission

1. Put all required files in a folder named with your student ID:

   ```
   Student_ID/
       source code (.cpp, .h)
       Makefile
       readme.txt          (how to build and run your program)
   ```

2. Archive the folder in a Linux environment; the archive name must be your student ID. For example, student 310510161 submits `310510161/` containing `Lab3.cpp`, `Makefile`, and `readme.txt`, archived with

   ```
   tar cvf 310510161.tar 310510161
   ```

3. Upload `Student_ID.tar` to e3. A submission in any other format receives 0 points. No written report is required.

- Deadline: **2026/11/23 23:59**. Late submissions lose 20% of the score per day, and any part of a day counts as a full day: a submission by 2026/11/24 23:59 loses 20%, by 2026/11/25 23:59 loses 40%, and so on.
- You may use AI tools, but you must write and submit your own code. All submissions are checked for similarity; if two are highly similar, both receive 0 points after confirmation.